

Curso Introducción a la Estadística con R

Clase 3: Introducción a la programación en R

Directorios de trabajo y proyectos de R (.Rproj)

Directorios de trabajo

- Para abrir en R un archivo guardado en tu computadora, debes especificar en qué carpeta está guardado, para esto hay varias opciones. Primero, puedes fijar un directorio por defecto:

```
# Puedo fijar el directorio de trabajo con la función setwd()
# Fijar la carpeta donde vamos a importar y exportar los archivos:
setwd("micompu/micarpeta")
getwd() # Con esta función puedo consultar el directorio
```

```
# Ahora, si quiero leer un archivo que esté en "micompu/micarpeta" simplemente
# escribo su nombre dentro de la función, en el lugar del "path".
```

```
# Supongamos que tengo dentro de la carpeta "micarpeta" un excel con datos
# de desempleo en Uruguay:
```

```
library(readxl)
desempleo_uru <- read_excel("data/desempleo.xlsx")
head(desempleo_uru, 4)
```

```
## # A tibble: 4 × 2
##   Year  tasa
##   <dbl> <dbl>
## 1  1990   8.5
## 2  1991   8.9
## 3  1992    9
## 4  1993   8.3
```

Directorios de trabajo

También podemos no fijar un directorio para la sesión e ir especificando los directorios completos dentro de cada función:

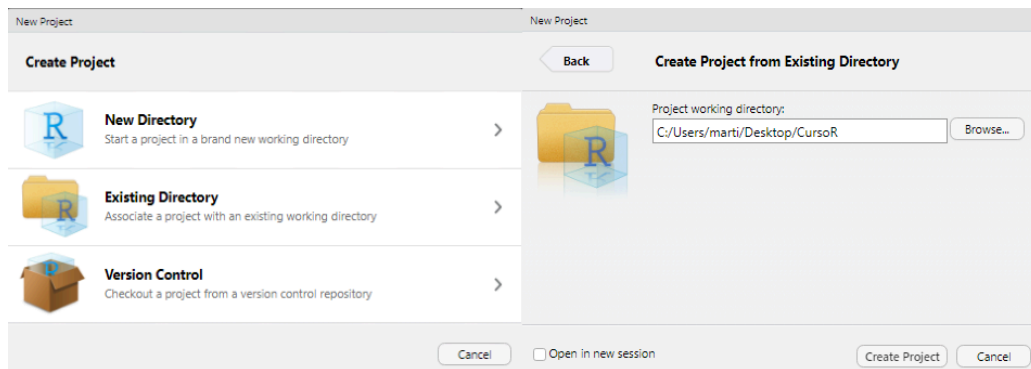
```
desempleo_uuru <- read_excel("micompu/micarpeta/data/desempleo.xlsx")
```

Proyectos de R (.Rproj)

- La mejor práctica para que nuestros scripts sean portables y reproducibles, es utilizar R Projects (.Rproj).
- Para crear un .Rproj vamos a `File/New Project` y ahí nos encontramos con la opción de crear una carpeta para guardar los archivos o utilizar una carpeta ya existente.
- Al crear un proyecto de R se creará un archivo de extensión .Rproj, cuando le damos click se inicia una nueva sesión de R cuyo directorio es por defecto la carpeta en la que está guardado.
- Podemos usar directorios relativos dentro de la carpeta en la que se aloja nuestro .Rproj para importar y exportar datos a y desde R. Esto hace que uno pueda cambiar la carpeta o compartirla y el script correrá de igual manera (a diferencia de si utilizamos `setwd()`)

Proyectos de R (.Rproj)

1. Creen una carpeta para guardar el material del curso
2. Descarga del EVA del curso las carpetas "Data" y las carpetas "scripts" de los días 1 y 2. Pegalas en la carpeta del punto 1 (guarda todos los scripts dentro de la misma carpeta)
3. Crea además dos carpetas más dentro de tu carpeta: "graficos" y "resultados"
4. Abran RStudio y desde ahí creen un *.Rproj*, desde existing directory, seleccionando la carpeta del punto 1



Proyectos de R (.Rproj)

5. Deberían ver en su carpeta algo así:

> CursorR

☐ Name	Date modified	Type	Size
 data	15/07/2021 18:06	File folder	
 graficos	15/07/2021 18:06	File folder	
 resultados	15/07/2021 18:06	File folder	
 scripts	15/07/2021 18:06	File folder	
 CursorR	15/07/2021 18:07	R Project	1 KB

Proyectos de R (.Rproj)

6. Abran el archivo `.Rproj` y desde ahí usando File/Open File abren los scripts dentro de la carpeta "scripts". Es importante que los abran desde la sesión que inicia el proyecto y no directamente haciendo click en el script.
7. Ya estamos listos para empezar!

Dialectos

Ejercicio

Supongamos que tengo estos datos:

```
## — Attaching core tidyverse packages ————— tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
## ✓ forcats    1.0.0      ✓ stringr   1.5.1
## ✓ ggplot2    3.5.0      ✓ tibble    3.2.1
## ✓ lubridate  1.9.3      ✓ tidyr     1.3.1
## ✓ purrr      1.0.2
## — Conflicts ————— tidyverse_conflicts() —
## ✗ dplyr::filter()      masks stats::filter()
## ✗ dplyr::group_rows()  masks kableExtra::group_rows()
## ✗ dplyr::lag()          masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
##
## Attaching package: 'data.table'
##
##
## The following objects are masked from 'package:lubridate':
##
##   hour, isoweek, mday, minute, month, quarter, second, wday, week,
##   yday, year
##
## The following objects are masked from 'package:dplyr':
##
##   between, first, last
##
## The following object is masked from 'package:purrr':
##
##   transpose
```

Ejercicio

Supongamos que tengo estos datos:

data

```
## # A tibble: 35 × 5
##   year      gdp_lcu inflation unemployment presidente
##   <dbl>      <dbl>      <dbl>      <dbl> <chr>
## 1  1985 249277574100      72.2         NA Sanguinetti
## 2  1986 271238450500      76.4         NA Sanguinetti
## 3  1987 292918912000      63.6         NA Sanguinetti
## 4  1988 297256857900      62.2         NA Sanguinetti
## 5  1989 300538279400      80.4         NA Sanguinetti
## 6  1990 301431925100     113.         NA Lacalle
## 7  1991 312099023700     102.         9.01 Lacalle
## 8  1992 336853433700      68.5         8.98 Lacalle
## 9  1993 345805469000      54.1         8.94 Lacalle
## 10 1994 370984750100      44.7          9 Lacalle
## # i 25 more rows
```

Ejercicio

¿Qué quiero hacer con el código debajo?

```
as.data.frame(t(sapply(X = split(
  x = data[which(data$presidente %in% c("Vázquez", "Sanguinetti")),
    which(colnames(data) %in% c("gdp_lcu", "inflation"))],
  f = data$presidente[which(data$presidente %in% c("Vázquez", "Sanguinetti"))],
  drop = TRUE),
  FUN = function(x) {apply(x, 2, mean)})))
```

Ejercicio

¿Qué quiero hacer con el código debajo?

```
data_dt <- data
setDT(data_dt)
data_dt[presidente %in% c("Vázquez", "Sanguinetti"),
  c("presidente", "gdp_lcu", "inflation") ][
  , lapply(.SD, mean), by = presidente]
```

Ejercicio

¿Qué quiero hacer con el código debajo?

```
data %>%  
  filter(presidente %in% c("Vázquez", "Sanguinetti")) %>%  
  select(presidente, gdp_lcu, inflation) %>%  
  group_by(presidente) %>%  
  summarise_all(mean)
```

R Base

```
as.data.frame(t(sapply(X = split(
  x = data[which(data$presidente %in% c("Vázquez", "Sanguinetti")),
    which(colnames(data) %in% c("gdp_lcu", "inflation"))],
  f = data$presidente[which(data$presidente %in% c("Vázquez", "Sanguinetti"))],
  drop = TRUE),
  FUN = function(x) {apply(x, 2, mean)})))
```

```
##                gdp_lcu inflation
## Sanguinetti 344923753631 46.168819
## Vázquez      584455715198  7.416316
```

Data.table

```
data_dt <- data
setDT(data_dt)
data_dt[presidente %in% c("Vázquez", "Sanguinetti"),
        c("presidente", "gdp_lcu", "inflation") ][
        , lapply(.SD, mean), by = presidente]
```

```
##      presidente      gdp_lcu inflation
##      <char>         <num>      <num>
## 1: Sanguinetti 344923753631 46.168819
## 2:   Vázquez  584455715198  7.416316
```



```
data %>%  
  filter(presidente %in% c("Vázquez", "Sanguinetti")) %>%  
  select(presidente, gdp_lcu, inflation) %>%  
  group_by(presidente) %>%  
  summarise_all(mean)
```

```
## # A tibble: 2 × 3  
##   presidente      gdp_lcu inflation  
##   <chr>          <dbl>    <dbl>  
## 1 Sanguinetti 344923753631.    46.2  
## 2 Vázquez    584455715198.     7.42
```

Dialectos

- En R podemos realizar una misma operación de muchas maneras distintas. Puesto de otra manera, R como lenguaje de programación tiene distintos "dialectos", esto es, paquetes (o conjuntos de paquetes) con sus propias funciones, sintaxis y comunidad de usuarios.
- Para la mayoría de las funciones requeridas para un análisis de datos estándar (importar datos, manipular, modelar y visualizar) existen -de forma muy simplificada- tres grandes dialectos: [R Base](#), [tidyverse](#) y [data.table](#).
- Tidyverse es una colección de paquetes diseñados para el análisis de datos. Este conjunto de paquetes comparte una filosofía de diseño, gramática y estructura de datos.
- Las ventajas de Tidyverse están en su gramática (fácil de leer lo que invita a compartir y replicar), consistencia, alcance y su numerosa y creciente comunidad.

Dialectos (ejemplo)

```
encuesta # Retomemos el data.frame "encuesta"
```

```
##   edad ideologia      voto
## 1   18 Izquierda Partido A
## 2   24 Izquierda Partido A
## 3   80  Derecha Partido C
```

```
# Supongamos que quiero quedarme solo con las variables de edad y voto
```

```
encuesta_base <- encuesta[ , c("edad", "voto")] # R Base
colnames(encuesta_base)
```

```
## [1] "edad" "voto"
```

```
encuesta_dt <- as.data.table(encuesta)[ , .(edad, voto)] # Datatable
colnames(encuesta_dt)
```

```
## [1] "edad" "voto"
```

```
encuesta_tidy <- select(encuesta, edad, voto) # Tidyverse
colnames(encuesta_tidy)
```

```
## [1] "edad" "voto"
```

Tidyverse

Tidyverse

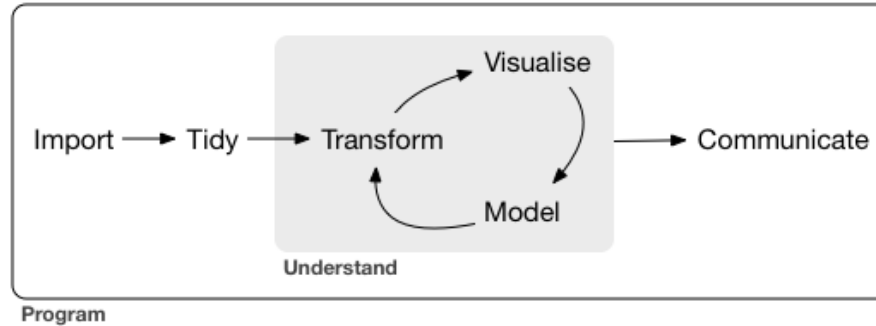
Tidyverse cuenta con varios paquetes que sirven para distintos tipos de tareas específicas. Podemos cargar todos los paquetes de forma conjunta:

```
# install.packages("tidyverse")  
library(tidyverse)  
  
# install.packages("dplyr")  
library(dplyr)
```



Tidyverse

La mejor manera de entender los principios de tidyverse es a través del libro del creador de tidyverse (Hadley Wickham) y Garrett Grolemund "R for Data Science" (2018).



Tidyverse: paquetes

Paquetes que Tidyverse carga:

- **readr**: importar y exportar datos
- **dplyr**: manipulación de datos
- **tidyr**: manipulación de datos
- **ggplot2**: visualización de datos
- **purrr**: programación avanzada
- **tibble**: estructura de datos
- **forcats**: factores
- **stringr**: variables de caracteres



Tidyverse: paquetes

Estos son algunos paquetes (para tareas más específicas) que forman parte del Tidyverse pero se tienen que cargar por separado:

- `readxl`: importar datos (excel)
- `haven`: importar (Stata, SPSS, SAS)
- `lubridate`: manipulación de fechas
- `rvest`: webscrapping
- `glue`: combinar data
- `tidymodels`: modelar datos

Importar y exportar datos

Importar datos

- Hasta ahora trabajamos principalmente con datos ingresados manualmente con las funciones `c()` y `data.frame()`
- Normalmente cuando trabajamos con datos solemos utilizar datos ya creados guardados en los formatos de otros programas (ej. Excel, Stata, SPSS)
- Existen varios paquetes que permiten importar y exportar datos desde distintos formatos. Algunos de los más utilizados son `readr`, `haven`, `readxl` y `utils`

Importar datos

Distintas funciones nos sirven para importar datos a R desde distintos formatos. Veamos algunos ejemplos:

```
# Con la función read_csv() del paquete readr importamos archivos .csv
library(tidyverse)
gapminder_csv <- read_csv("data/gapminder.csv")

# Con la función read_excel() del paquete readxl importamos archivos excel
library(readxl)
gapminder_excel <- read_excel("data/gapminder.xlsx")
```

```
# Vemos que los dataframes son iguales, tienen la mismas filas y columnas
dim(gapminder_csv)
```

```
## [1] 1704    6
```

```
dim(gapminder_excel)
```

```
## [1] 1704    6
```

Importar datos

Algunos paquetes incluyen datos, por ejemplo, gapminder. En la documentación del paquete se encuentra el nombre de los datos. Con una simple asignación los podemos cargar

```
library(gapminder)

data_gapminder <- gapminder
head(data_gapminder)
```

```
## # A tibble: 6 × 6
##   country      continent  year lifeExp      pop gdpPercap
##   <fct>        <fct>    <int>  <dbl>    <int>    <dbl>
## 1 Afghanistan Asia      1952   28.8  8425333    779.
## 2 Afghanistan Asia      1957   30.3  9240934    821.
## 3 Afghanistan Asia      1962   32.0 10267083    853.
## 4 Afghanistan Asia      1967   34.0 11537966    836.
## 5 Afghanistan Asia      1972   36.1 13079460    740.
## 6 Afghanistan Asia      1977   38.4 14880372    786.
```

Importar datos

También es posible importar datos guardados en los formatos de otros softwares estadísticos como SPSS o Stata. Para esto usaremos el paquete haven.

```
library(haven)

# SPSS
gapminder_spss <- read_spss("data/gapminder.sav")

# STATA
gapminder_stata <- read_stata("data/gapminder.dta")
```

O podríamos llamar a la función y paquete dado que generalmente solo utilizamos una función de los paquetes que cargan datos (depende del caso obviamente)

```
# SPSS
gapminder_spss <- haven::read_spss("data/gapminder.sav")

# STATA
gapminder_stata <- haven::read_stata("data/gapminder.dta")
```

Importar datos

R también cuenta con sus propios formatos de almacenamiento de datos (`.rds` y `.Rdata` o `.rda`). Este enfoque es poco práctico si queremos usar los datos almacenados en otro programa, pero muy útil si solamente usaremos R dado que mantiene la información tal cual estaba en R (por ej. tipos de variables o atributos):

```
# Para esto no necesitamos cargar paquetes.
# Guardar un objeto como .rds:
saveRDS(object = data_gapminder,
        file = "resultados/data_gapminder.rds")

# Leemos un archivo .rds
miobjeto_rds <- readRDS(file = "resultados/data_gapminder.rds")

# Con .rda se pueden guardar varios objetos al mismo tiempo!
# Exportamos un archivo .Rdata
save(data_gapminder, miobjeto_rds,
     file = "resultados/dos_dataframes.Rdata")

# Importamos un archivo .Rdata
load("resultados/dos_dataframes.Rdata")
```

Exportar datos

- También podemos guardar archivos desde R en otros formatos.
- Con `readr` podemos exportar archivos en formato .csv
- Con `writexl` podemos exportar directamente un excel.
- Con `haven` podemos exportar archivos en formato .dta (Stata) y .sav (SPSS)

Importar y exportar datos

Argumentos a tener en cuenta:

- **Nombre de columnas:** a veces debemos especificar si queremos que la primera fila de nuestros datos sean el nombre de las variables
- **Nombre de filas:** de igual manera, a veces podemos especificar si queremos que la primera columna sea el nombre de las filas (sirve para identificadores de caso por ej.)
- **Etiquetas de variables:** cuando los datos que queremos importar tienen etiquetas (pasa mucho en encuestas) podemos cargarlas como etiquetas o cargar solamente la etiqueta como cadena o factores. Ver capítulo 4 de Urdinez, F. & Labrin, A. (Eds.) (2020)
- **Append:** algunas funciones permiten agregar filas debajo de un archivo (esto es muy útil para ir actualizando bases de datos)

Etiquetas cuando importamos datos

- Cuando importamos datos que tienen etiquetas (por ejemplo de formatos como Stata o SPSS) debemos tener cuidado con cómo manejar estas etiquetas
- Por ejemplo, supongamos que queremos leer los datos de una encuesta con dos variables, guardada en formato Stata (`.dta`), con el paquete `haven`.
- Por defecto se leen como variables de tipo `double` (numérica) con etiquetas como atributos
- Si queremos quedarnos directamente con las etiquetas, podemos utilizar la función `as_factor`.

Importar y exportar datos: factores

- Otro tipo de variables en R son los factores (factors), utilizados para representar data categórica. Estos suelen confundirse con las variables de caracteres pero tienen algunas diferencias.
- Normalmente los factores son utilizados para las variables de caracteres con un número de valores posibles fijo y cierto orden (opcional)
- A R le gusta transformar las variables de caracteres en factores al importarlas (si usamos R Base particularmente).
- El paquete `forcats` (dentro del Tidyverse) ayuda a manejar variables de caracteres y factores:
 - `fct_relevel()` cambia manualmente el orden de los niveles
 - `fct_reorder()` cambia el orden de los niveles de acuerdo a otra variable
 - `fct_infreq()` reordena un factor por la frecuencia de sus valores
 - `fct_lump()` collapse los valores menos frecuentes en otra categoría "other". Es muy útil para preparar datos para tablas y gráficos

Importar y exportar datos: factores

```
# Podemos chequear y coercionar factores
data_gapminder <- gapminder
is.factor(data_gapminder$continent) # Chequeo si es factor
```

```
## [1] TRUE
```

```
levels(data_gapminder$continent) # Chequeo los niveles
```

```
## [1] "Africa" "Americas" "Asia" "Europe" "Oceania"
```

```
# Transformo a caracter
data_gapminder$continent <- as.character(data_gapminder$continent)
class(data_gapminder$continent)
```

```
## [1] "character"
```

```
# De vuelta a factor
data_gapminder$continent <- as.factor(data_gapminder$continent)
class(data_gapminder$continent)
```

```
## [1] "factor"
```

Importar y exportar datos: factores

```
# Para crear un factor usamos la función factor()  
países_mercosur <- factor(c("Argentina", "Brasil", "Paraguay", "Uruguay"))  
table(países_mercosur)
```

```
## países_mercosur  
## Argentina      Brasil   Paraguay   Uruguay  
##           1           1           1           1
```

```
# La función fct_relevel() nos permite reordenar los niveles del factor  
países_mercosur <- fct_relevel(países_mercosur, "Uruguay")  
table(países_mercosur)
```

```
## países_mercosur  
##   Uruguay Argentina      Brasil   Paraguay  
##         1           1           1           1
```

Ejercicio

- (1) En la carpeta data encontrarán un archivo excel llamado "urudata_sheets", leer el archivo en R*
- (2) Leer la segunda hoja del archivo*

Dataframes

Dataframes: tibbles

- La mayoría de los análisis de datos convencionales contienen dataframes. Cuando usamos los paquetes del tidyverse, generalmente trabajamos con "tibbles", que es muy similar a un dataframe pero con pequeños cambios.
- Una de las principales diferencias es la forma en que se imprimen los datos.
- La mayoría de las funciones del Tidyverse devuelven un tibble.

```
data_gapminder <- (gapminder)
class(data_gapminder) # Ya es un tibble
```

```
## [1] "tbl_df"      "tbl"        "data.frame"
```

```
data_gapminder <- as.data.frame(data_gapminder)
class(data_gapminder) # Ahora solamente dataframe
```

```
## [1] "data.frame"
```

Dataframes: imprimir dataframe

```
print(data_gapminder)
```

	country	continent	year	lifeExp	pop	gdpPercap
## 1	Afghanistan	Asia	1952	28.80100	8425333	779.4453
## 2	Afghanistan	Asia	1957	30.33200	9240934	820.8530
## 3	Afghanistan	Asia	1962	31.99700	10267083	853.1007
## 4	Afghanistan	Asia	1967	34.02000	11537966	836.1971
## 5	Afghanistan	Asia	1972	36.08800	13079460	739.9811
## 6	Afghanistan	Asia	1977	38.43800	14880372	786.1134
## 7	Afghanistan	Asia	1982	39.85400	12881816	978.0114
## 8	Afghanistan	Asia	1987	40.82200	13867957	852.3959
## 9	Afghanistan	Asia	1992	41.67400	16317921	649.3414
## 10	Afghanistan	Asia	1997	41.76300	22227415	635.3414
## 11	Afghanistan	Asia	2002	42.12900	25268405	726.7341
## 12	Afghanistan	Asia	2007	43.82800	31889923	974.5803
## 13	Albania	Europe	1952	55.23000	1282697	1601.0561
## 14	Albania	Europe	1957	59.28000	1476505	1942.2842
## 15	Albania	Europe	1962	64.82000	1728137	2312.8890
## 16	Albania	Europe	1967	66.22000	1984060	2760.1969
## 17	Albania	Europe	1972	67.69000	2263554	3313.4222
## 18	Albania	Europe	1977	68.93000	2509048	3533.0039
## 19	Albania	Europe	1982	70.42000	2780097	3630.8807
## 20	Albania	Europe	1987	72.00000	3075321	3738.9327
## 21	Albania	Europe	1992	71.58100	3326498	2497.4379
## 22	Albania	Europe	1997	72.95000	3428038	3193.0546
## 23	Albania	Europe	2002	75.65100	3508512	4604.2117
## 24	Albania	Europe	2007	76.42300	3600523	5937.0295
## 25	Algeria	Africa	1952	43.07700	9279525	2449.0082
## 26	Algeria	Africa	1957	45.68500	10270856	3013.9760
## 27	Algeria	Africa	1962	48.30300	11000948	2550.8169
## 28	Algeria	Africa	1967	51.40700	12760499	3246.9918
## 29	Algeria	Africa	1972	54.51800	14760787	4182.6638
## 30	Algeria	Africa	1977	58.01400	17152804	4910.4168
## 31	Algeria	Africa	1982	61.36800	20033753	5745.1602
## 32	Algeria	Africa	1987	65.79900	23254956	5681.3585
## 33	Algeria	Africa	1992	67.74400	26298373	5023.2166
## 34	Algeria	Africa	1997	69.15200	29072015	4797.2951
## 35	Algeria	Africa	2002	70.99400	31287142	5288.0404
## 36	Algeria	Africa	2007	72.30100	33333216	6223.3675
## 37	Angola	Africa	1952	30.01500	4232095	3520.6103
## 38	Angola	Africa	1957	31.99900	4561361	3827.9405
## 39	Angola	Africa	1962	34.00000	4826015	4269.2767
## 40	Angola	Africa	1967	35.98500	5247469	5522.7764
## 41	Angola	Africa	1972	37.92800	5894858	5473.2880
## 42	Angola	Africa	1977	39.48300	6162675	3008.6474
## 43	Angola	Africa	1982	39.94200	7016384	2756.9537
## 44	Angola	Africa	1987	39.90600	7874230	2430.2083

Dataframes: imprimir tibble

```
data_gapminder <- as_tibble(data_gapminder) # Pasamos nuevamente a tibble
class(data_gapminder)
```

```
## [1] "tbl_df"      "tbl"        "data.frame"
```

```
print(data_gapminder)
```

```
## # A tibble: 1,704 × 6
##   country      continent year lifeExp      pop gdpPercap
##   <fct>        <fct>    <int>   <dbl>    <int>    <dbl>
## 1 Afghanistan Asia      1952    28.8  8425333    779.
## 2 Afghanistan Asia      1957    30.3  9240934    821.
## 3 Afghanistan Asia      1962    32.0 10267083    853.
## 4 Afghanistan Asia      1967    34.0 11537966    836.
## 5 Afghanistan Asia      1972    36.1 13079460    740.
## 6 Afghanistan Asia      1977    38.4 14880372    786.
## 7 Afghanistan Asia      1982    39.9 12881816    978.
## 8 Afghanistan Asia      1987    40.8 13867957    852.
## 9 Afghanistan Asia      1992    41.7 16317921    649.
## 10 Afghanistan Asia      1997    41.8 22227415    635.
## # i 1,694 more rows
```

Tidy dataset

- Hay muchas formas de estructurar un conjunto de datos. El enfoque tidy sugiere que cada variable sea una columna y cada observación sea una fila, por lo que cada valor tiene su propia celda:

country	year	cases	population
Afghanistan	1999	745	15987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	17206362
Brazil	2000	80488	174604898
China	1999	212258	1272915272
China	2000	216766	128042583

variables

country	year	cases	population
Afghanistan	1999	745	15987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	17206362
Brazil	2000	80488	174604898
China	1999	212258	1272915272
China	2000	216766	128042583

observations

country	year	cases	population
Afghanistan	1999	745	15987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	17206362
Brazil	2000	80488	174604898
China	1999	212258	1272915272
China	2000	216766	128042583

values

Wickham & Golemund (2018)

Nombres de variables

Muchas veces usamos datos que no están documentados de manera uniforme o apropiada, por ejemplo, con nombres dispares y propensos a errores en las columnas.

Janitor es un paquete orientado al estilo Tidyverse (aunque no pertenece) que facilita algunas funciones para limpiar y explorar datos.

```
ejemplo
```

```
##   COLORES NombresCompletos edad_NUMERICA
## 1   Verde      María S.           32
## 2   Rojo       Juan F.            23
## 3   Azul       Pedro A.           24
```

```
library(janitor)
```

```
ejemplo_clean <- clean_names(ejemplo)
ejemplo_clean
```

```
##   colores nombres_completos edad_numerica
## 1   Verde      María S.           32
## 2   Rojo       Juan F.            23
## 3   Azul       Pedro A.           24
```